

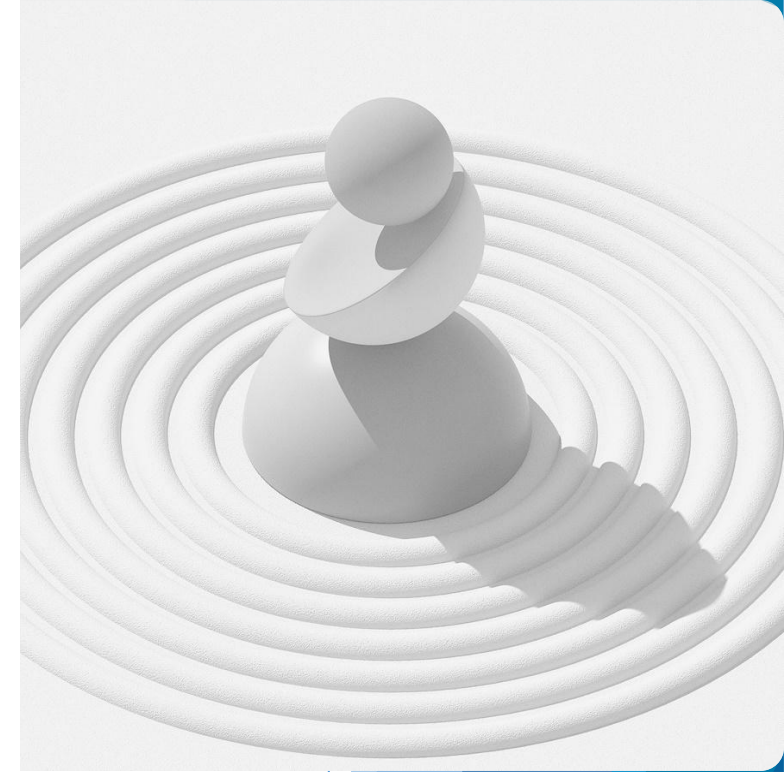
Presented by Nirvana Popescu

Lesson 3: Software design tools for Critical Software

Outline

- ▶ Why software design tools matter in Critical Software
- ▶ Tools Categories for Critical Software Engineering
- ▶ Scade One, a model-based software development tool
- ▶ Swan Synchronous Language
- ▶ Scade One / Swan - introduction
 - ▶ Project (modules, libraries(use-directives), jobs)
 - ▶ Data-types, Constants
 - ▶ Dataflow operators
 - ▶ Control flow operators

The course is based on [Getting started](#) tutorial from Ansys Scade one page. For more details, please consult it.



Why software design tools matter in Critical Software

- ▶ **1. Improved System Reliability**
 - ▶ Visual modeling (UML, SysML) reduces design errors
 - ▶ Early validation before implementation
- ▶ **2. Better Communication**
 - ▶ Clear diagrams help teams understand complex systems
 - ▶ Aligns developers, engineers, and stakeholders
- ▶ **3. Early Error Detection**
 - ▶ Simulation & modeling tools catch issues before deployment
 - ▶ Reduces costly fixes later
- ▶ **4. Standardization & Compliance**
 - ▶ Supports industry standards (e.g., safety regulations)
 - ▶ Easier certification processes
- ▶ **5. Complexity Management**
 - ▶ Breaks large systems into manageable components
 - ▶ Improves scalability and maintainability





Tools Categories for Critical Software Engineering

▶ 1. Requirements

- ▶ Specify systems and software behavior
- ▶ Examples: IBM Doors, Polarion, Reqtify

▶ 2. Architecture

- ▶ Create system structure and component interactions
- ▶ Support modular and scalable design
- ▶ Examples: UML, SysML

▶ 3. Model-Based Software Development

- ▶ Design, simulate, and generate code from software models
- ▶ Example: SCADE, Scade One, MathWorks Simulink



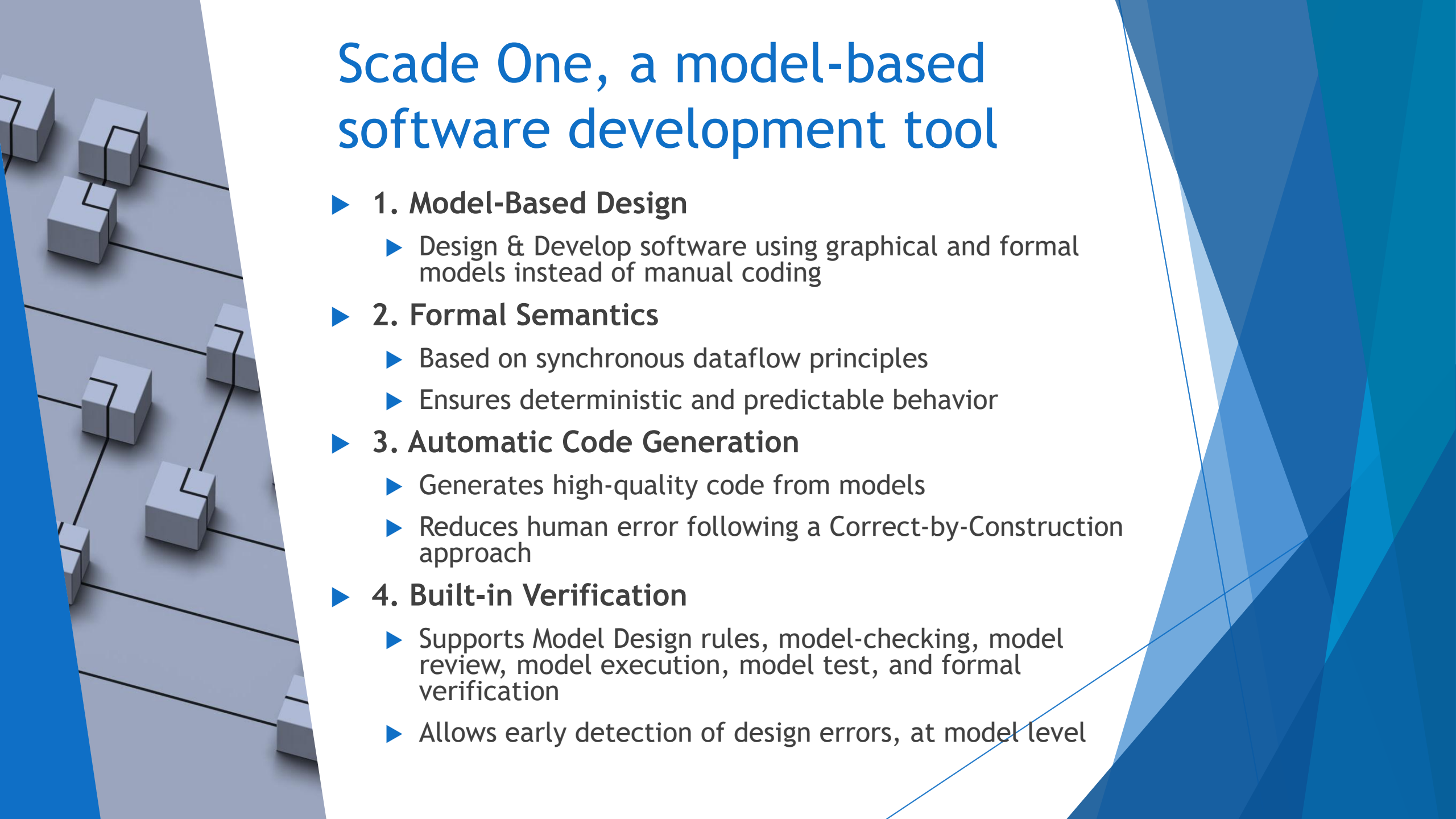
Tools Categories for Critical Software Engineering

▶ 4. Verification & Validation Tools

- ▶ Testing, simulation, and formal verification
- ▶ Ensure system correctness and safety
- ▶ Examples: Scade One, Synopsys TPT, Vector Cast, LDRA, RTRT,

▶ 5. Configuration & Version Control Tools

- ▶ Manage model and code changes and versions
- ▶ Track updates and maintain consistency
- ▶ Examples: Github, SVN



Scade One, a model-based software development tool

- ▶ **1. Model-Based Design**
 - ▶ Design & Develop software using graphical and formal models instead of manual coding
- ▶ **2. Formal Semantics**
 - ▶ Based on synchronous dataflow principles
 - ▶ Ensures deterministic and predictable behavior
- ▶ **3. Automatic Code Generation**
 - ▶ Generates high-quality code from models
 - ▶ Reduces human error following a Correct-by-Construction approach
- ▶ **4. Built-in Verification**
 - ▶ Supports Model Design rules, model-checking, model review, model execution, model test, and formal verification
 - ▶ Allows early detection of design errors, at model level

Swan synchronous languages

► What is a Synchronous Language?

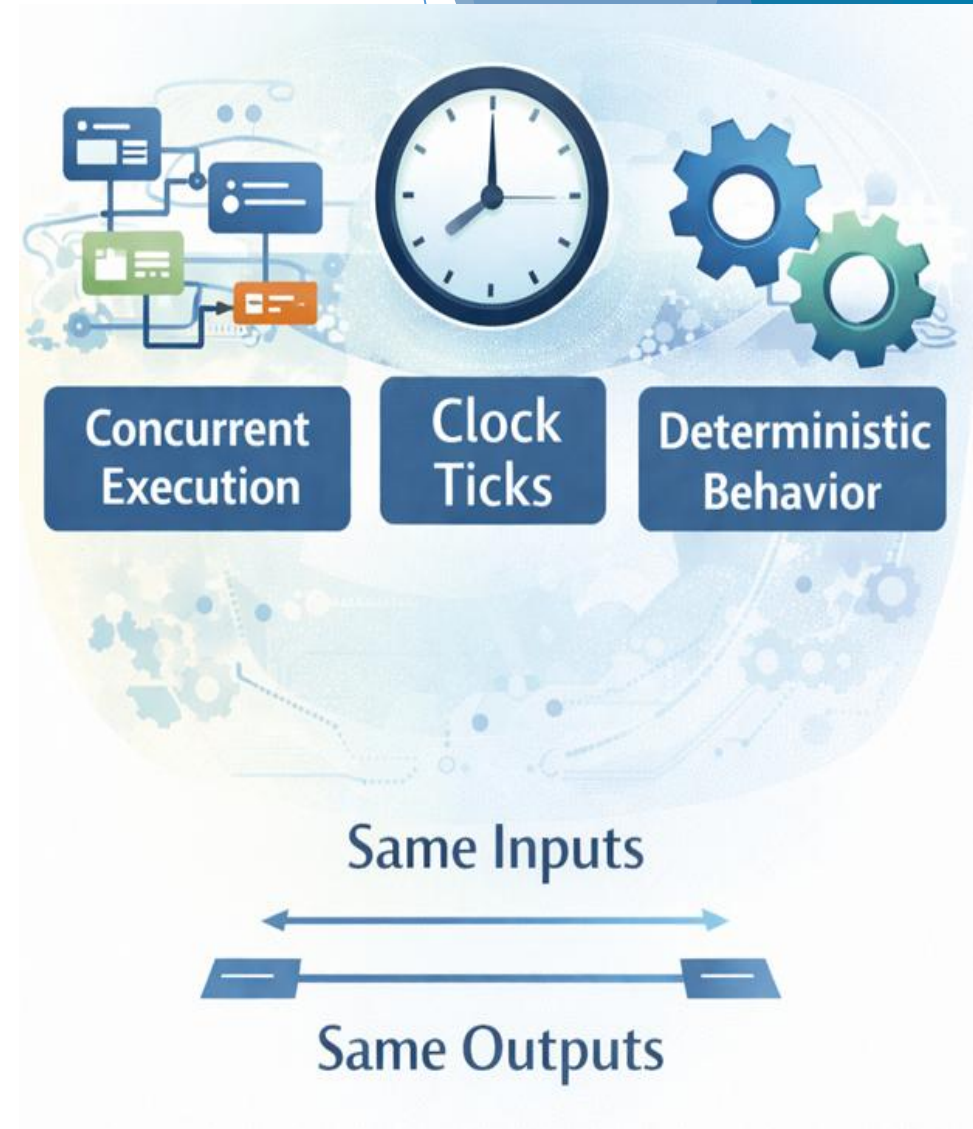
- Programs react in discrete logical steps (ticks)
- All operations occur simultaneously at each tick
- Deterministic behavior: same inputs → same outputs
- Abstracts logical time for easier reasoning

► Key Features:

- **Concurrency support:** multiple components run in sync
- **Formal verification-friendly:** predictable behavior allows mathematical analysis and proofs

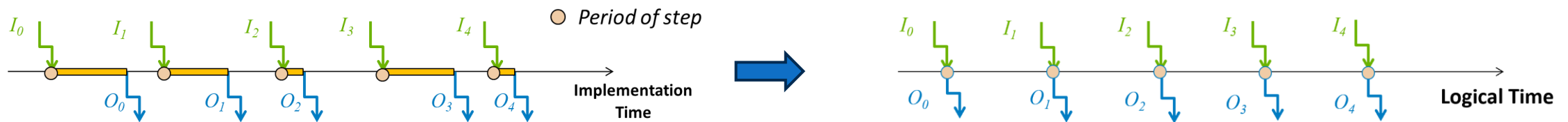
► Example:

- **Swan Language** - used in Scade One



Swan synchronous languages

Need	Response
Reaction = Reading/Calculating/Writing during a step	Infinite sequences of values over time (discrete mode = steps)
Process control application: $\begin{cases} x_t = f(x_{t-1}, y_{t-1}); y_t = g(x_{t-1}, y_{t-1}); \\ x_0 = 0; y_0 = 0 \end{cases}$	Discrete time expression: pre operator gives access to the previous time step (the "t-1")
Control/choice applications: management of modes, states, etc.	Automata and other decision structures
Runtime error: stackoverflow, out-of-bound,...	No dynamic allocation, static loops, no pointers
Determinism	Same input sequence produces same output sequence
Ability to reason formally about a program → mathematical model	a/ Synchronism assumption: - data produced in zero time - no execution overlapping b/ Formal definition of the language



Notion of Flow

- ▶ A flow = an **infinite sequence of typed values**, scalar or of a composite type (array / structure / variant)
- ▶ All input flows are evaluated before the calculation is performed
- ▶ A flow has a unique definition at each step
- ▶ Operations create new flows

Step \ Time	1	2	3	4	...
	1	2	3	4	...
3	3	3	3	3	...
a	a_1	a_2	a_3	a_4	...
b	b_1	b_2	b_3	b_4	...
a+b	a_1+b_1	a_2+b_2	a_3+b_3	a_4+b_4	...

The result is
available
within the
same cycle

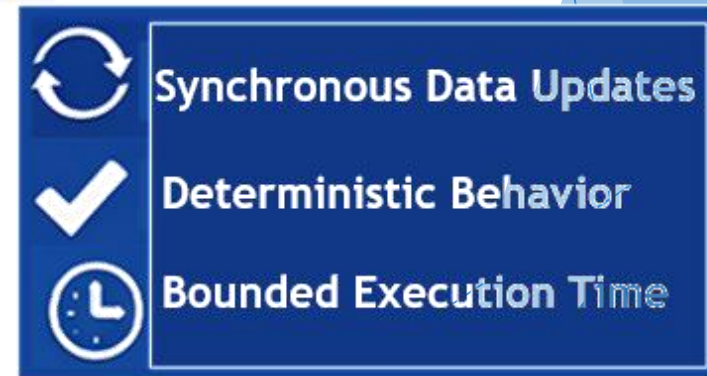
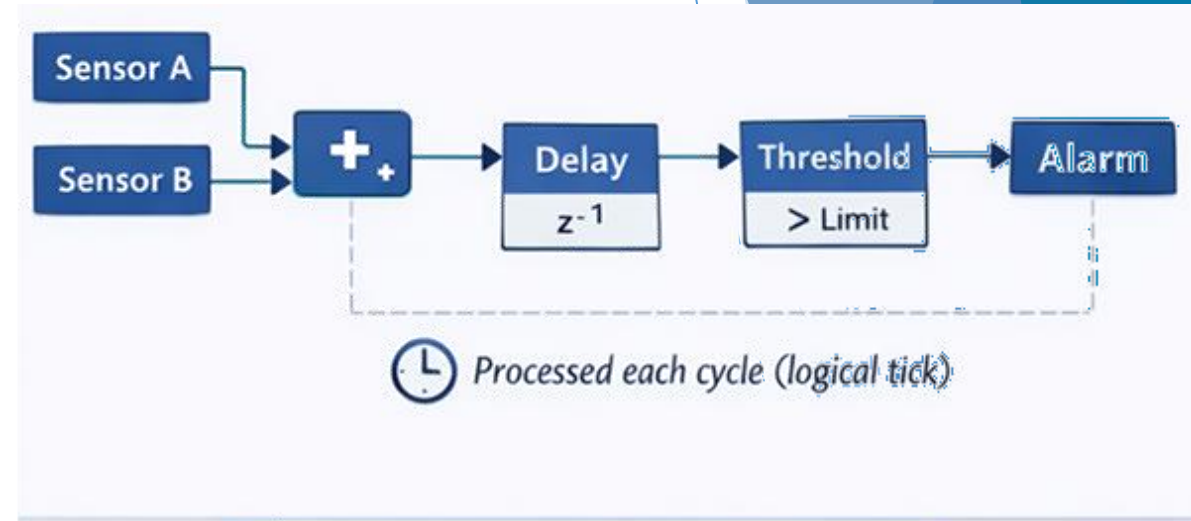
Swan Dataflow

► What is Dataflow?

- The software described as a network of nodes connected by data streams.
- Each node computes outputs from inputs every clock tick.
- Focuses on how data moves, not line-by-line code execution.
- Data-flows express mathematical equations

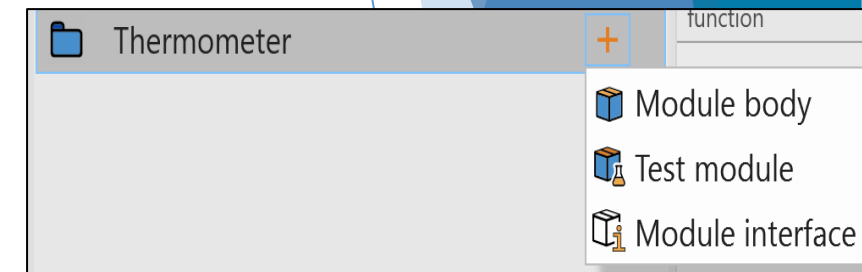
► Key Principles:

- **Synchronous execution:** all nodes update **simultaneously** per tick
- **Deterministic behavior:** same inputs → same outputs each tick
- **Logical time abstraction:** operations modeled as instantaneous at each tick
- **Concurrency by design:** independent nodes can run in parallel safely
- Data Dependencies between concurrent dataflows are resolved by the code generator



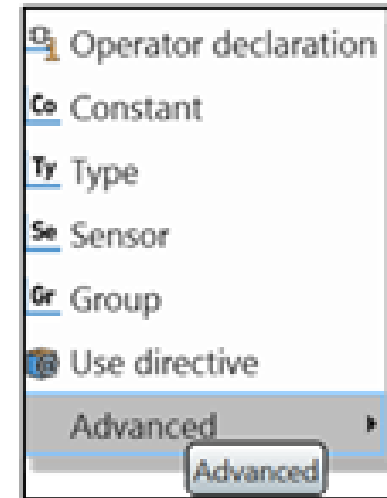
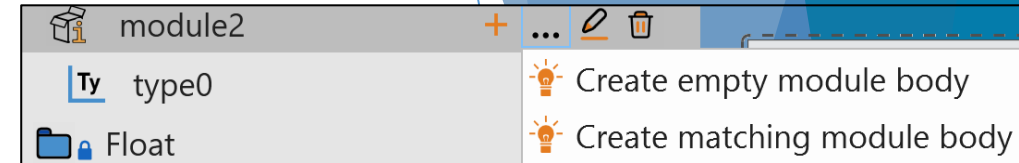
Scade One/Swan introduction

- ▶ When you create a project in Scade One, it's typically organized in modules and jobs.
 - ▶ **Module body:** contains the definition of elements declared in the module interface. It can contain elements that are not declared in the interface: these additional elements are private and so can only be used in the definitions within the module body (serialized as **.swan file**)
 - ▶ **Test module:** a module that extends the properties of the previous module to contain the definition of test harnesses (operators used for running model execution, serialized as **.swant file**)
 - ▶ **Module interface:** to declare all the types, constants and operators that are public and then usable by another module. Using interfaces is optional (serialized as **.swani file**).
 - ▶ **A job:** represents a **specific automated process or task** that the Scade one environment executes. It can be seen as:
 - ▶ A step in a pipeline: e.g., compile → generate code → run tests → package results.
 - ▶ A repeatable process for CI/CD (continuous integration / deployment)



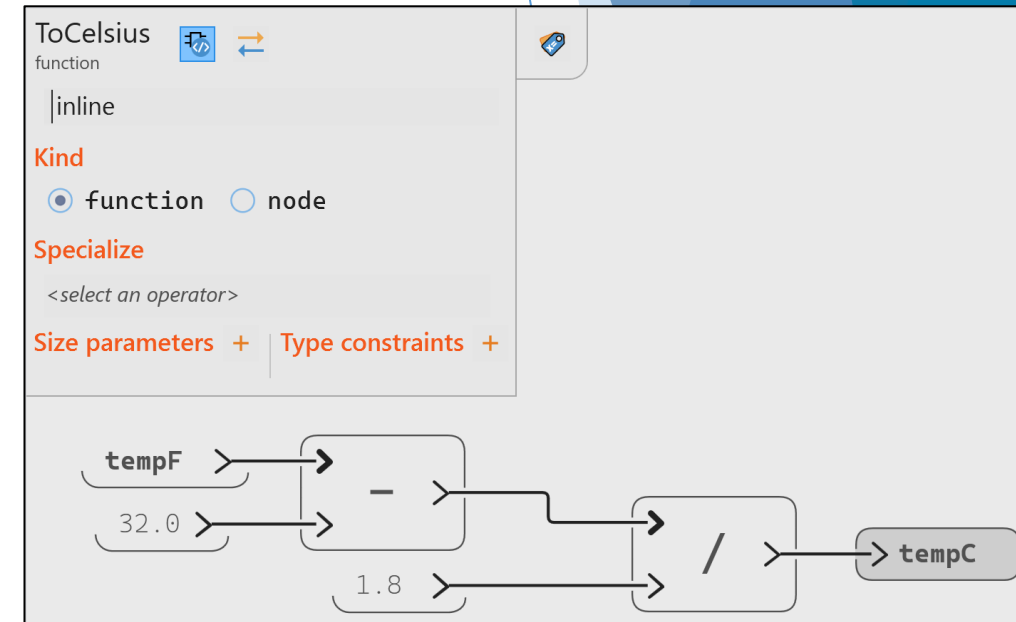
Module body

- ▶ Contains the definitions of the following elements:
 - ▶ **use directives:** to use elements from another module
 - ▶ **groups:** list of flows to gather several wires (to minimize the number of connections and so wires)
 - ▶ **types**
 - ▶ **constants**
 - ▶ **sensors**
 - ▶ **operator definitions and declarations** (graphical or textual)



Operator

- ▶ **Operator** = means of re-using and structuring a model
- ▶ It's a computing unit with inputs and outputs
- ▶ It can be graphical or textual. Graphical means that we define the operator outputs from its inputs using the diagrammatic style (graphical blocks) and/or equational style of the Swan language.
- ▶ Notion of substitution: an operator instance is replaceable by the equations that define it
 - ▶ Operator = relationship between inputs and outputs
 - ▶ Direct output/input loopback is forbidden
- ▶ A user -defined operator can be **node** or **function**
 - ▶ **function**: must contain only **combinational equations** (defining output flows only based on current input flows, no memory), including all instantiated (= called) operators
 - ▶ **node**: contains a memory (sequential operators such as **->**, **pre**, and **window**; **last**, **automaton**)



Swan Data-flow predefined operators

- ▶ The predefined operators are presented in the *User Documentation*:
 - ▶ Arithmetic operator (+, -, *, /, mod, numeric cast, negation -)
 - ▶ Bitwise operators (land, lor, lxor, lnot, logical shift left, logical shift right)
 - ▶ Relational operators (=, <>, >, <, >=, <=)
 - ▶ Boolean operators (logical or, xor, and, not)
 - ▶ Array operators and constructs (constructor, repetition, pack, projection, etc.)
 - ▶ Structure / variant operators (structure and variant constructors, structure destructor, etc.)
 - ▶ Time operators(sequential) operators (->, pre, window)

Time Operators

- Express the discrete time => for accessing the previous step and initialization

$$\begin{cases} x_t = f(x_{t-1}, y_{t-1}); \\ y_t = g(x_{t-1}, y_{t-1}); \\ x_0 = 0; y_0 = 0 \end{cases}$$

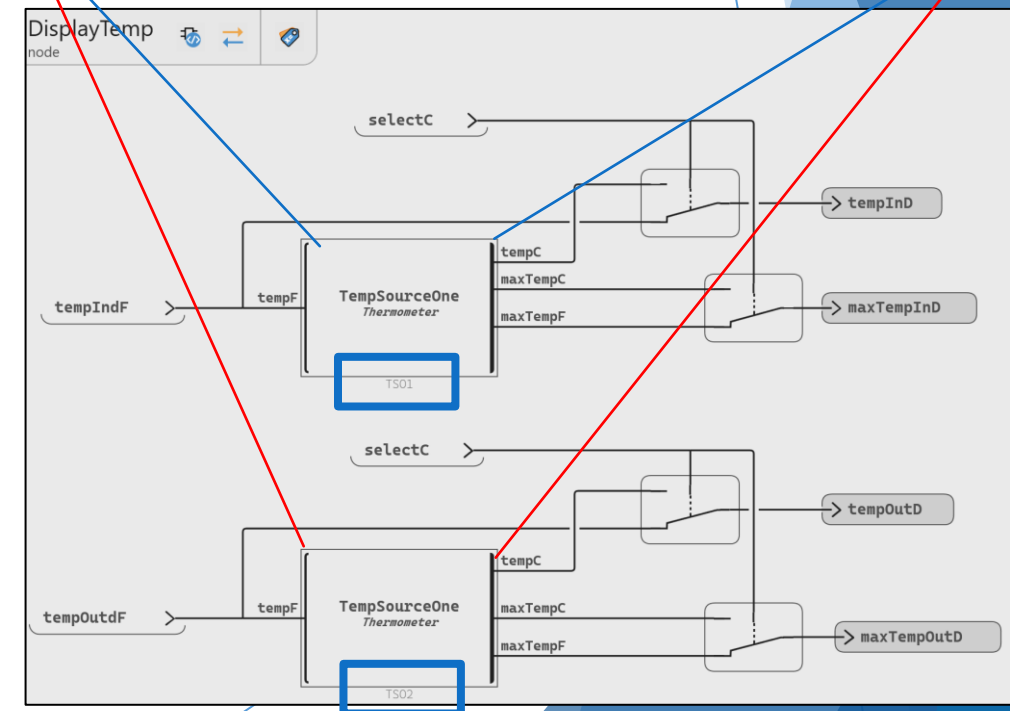
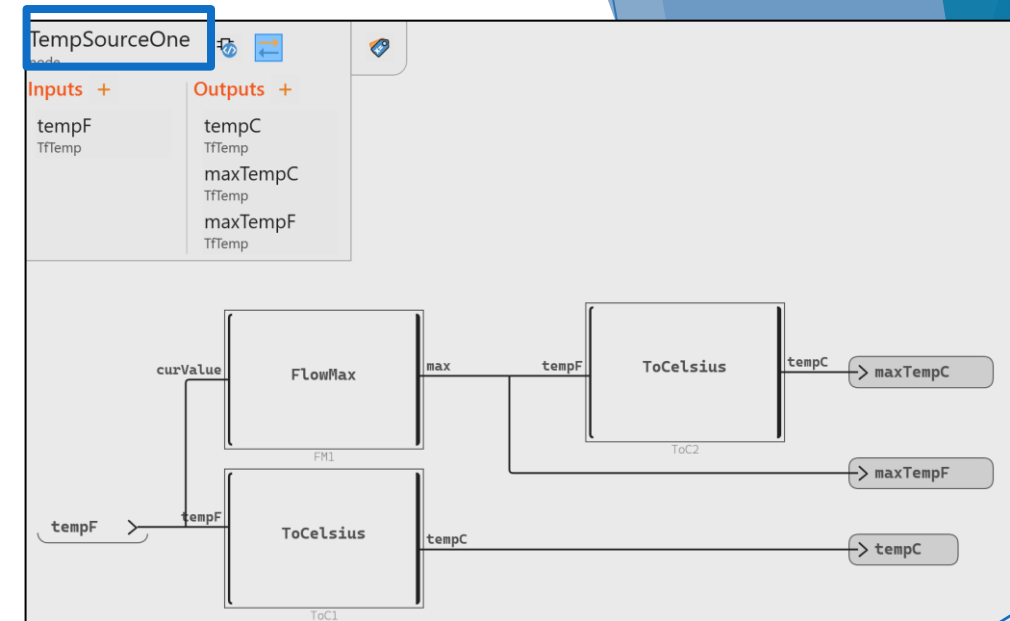
Time Step Data		1	2	3	4	5	...
Unit Delay	f	3	8	4	2	7	...
	pre f	undef	3	8	4	2	...
Initial Value	9 -> f	9	8	4	2	7	...
	4 -> pre f	4	3	8	4	2	

pre operator (**Unit Delay**) returns the value of the flow at the previous step (no initial value)

-> operator (**Initial Value**) sets the value of the flow at the first step

Operator Instance

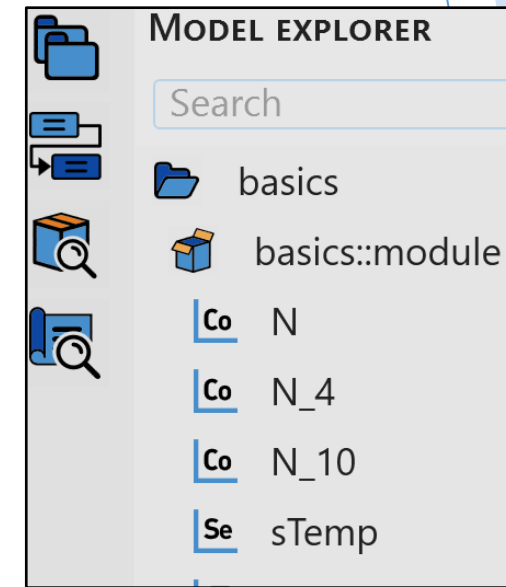
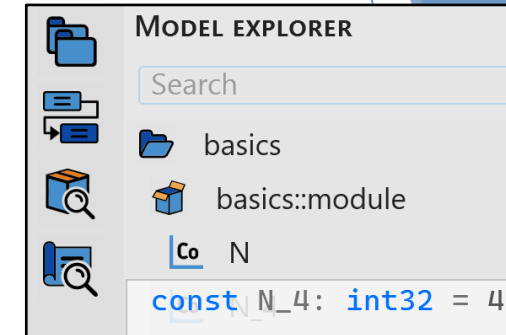
- ▶ Reusing a user operator or a predefined operator is called operator instantiation
- ▶ The reused operator is an instance of that operator. Several instances of the same operator can be introduced to factorize part of a design and structure a model.
- ▶ In a body of operator or block diagram, you can rename instances to distinguish them from each other.



Constant Declaration

- ▶ You can use constant values like 32.0 and [0.5,0.25,0.125,0.0625]
- ▶ They represent constant flows (they have the same value in each step).
- ▶ Constant declaration starts with the **const** keyword and is made of an identifier, a type expression, and a value expression. The value expression must be known at compile time

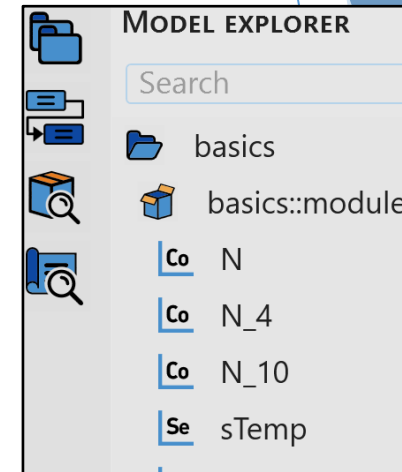
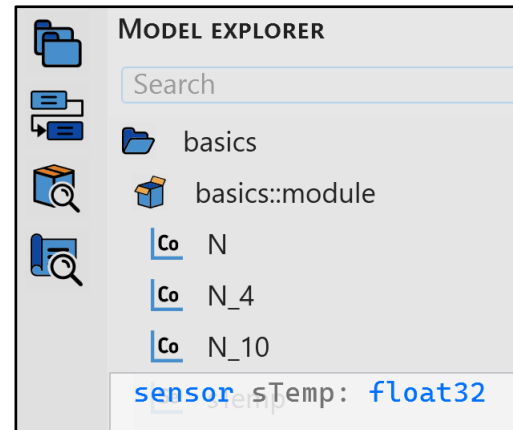
```
const N: uint32 = 1;  
const OFF: bool = false;  
const N_4: int32 = 4;  
const V_4: float64^N_4 = [0.5,0.25,0.125,0.0625];
```



Sensor Declaration

- ▶ You can declare sensors that represent global input flows
- ▶ Sensor declaration starts with the **sensor** keyword and is made of an identifier, and a type expression

```
sensor sTemp: float32;  
sensor sCoord2D: int16^2;
```



Swan Types Declaration

- ▶ The Swan language is a strongly typed language (any flow is typed) and **has no** automatic type conversion (cast):
 - ▶ An integer is not a real ($0 \neq 0.0$), a Boolean is not a non-zero integer
 - ▶ Conversions must be explicit: you can use the **cast** operator, to convert the flow
 - ▶ Typing analysis is based on type structure, not name
- ▶ The native Swan type system is built out of atomic data types (predefined types)
 - ▶ Predefined types are **float32**, **float64**, **bool**, **char**, signed **int8** to **int64** and unsigned **uint8** to **uint64**
 - ▶ These predefined types are used to declare complex types such as typedef, array, structure or variant types

```
type TfTemp = float32;  
type TaTemp = TfTemp^N_3;  
type Tboolean = bool;  
type TsCart = {x: int8; y: int16};  
sensor sTempvN: float32^N;
```

	types::module
<u>Ty</u>	TfTemp
<u>Ty</u>	TaTemp
<u>Ty</u>	Tboolean
<u>Ty</u>	TsCart

Swan Types Declaration

- ▶ Named Type declaration starts with the **type** keyword and is made of an identifier and:

- ▶ a type expression

```
type TfTemp = float32;
```

```
type TtempOut = Thermometer::TfTemp;
```

- ▶ an enumeration type

```
type TeMode = enum {OFF, ON, DAY, NIGHT};
```

- ▶ a structure type

```
type TsCart3D = {x: float64, y: float64, z: float32};
```

- ▶ a variant type

```
type TvCoord2D = Polar{r: float32, t: float32}  
                | Cartesian{x: float32, y: float32};
```

	types::module
	TfTemp
	TaTemp
	TtempOut
	Tboolean
	TsCart
	TsCart3D
	TsPolar
	TvCoord2D
	TeMode

For more details, please consult Scade one user documentations in Getting started from Ansys Scade one page

Swan Control flow

▶ Control flow:

- ▶ Describes how execution is directed (conditions, states, sequences)
- ▶ Complements dataflow by managing when and which operations run

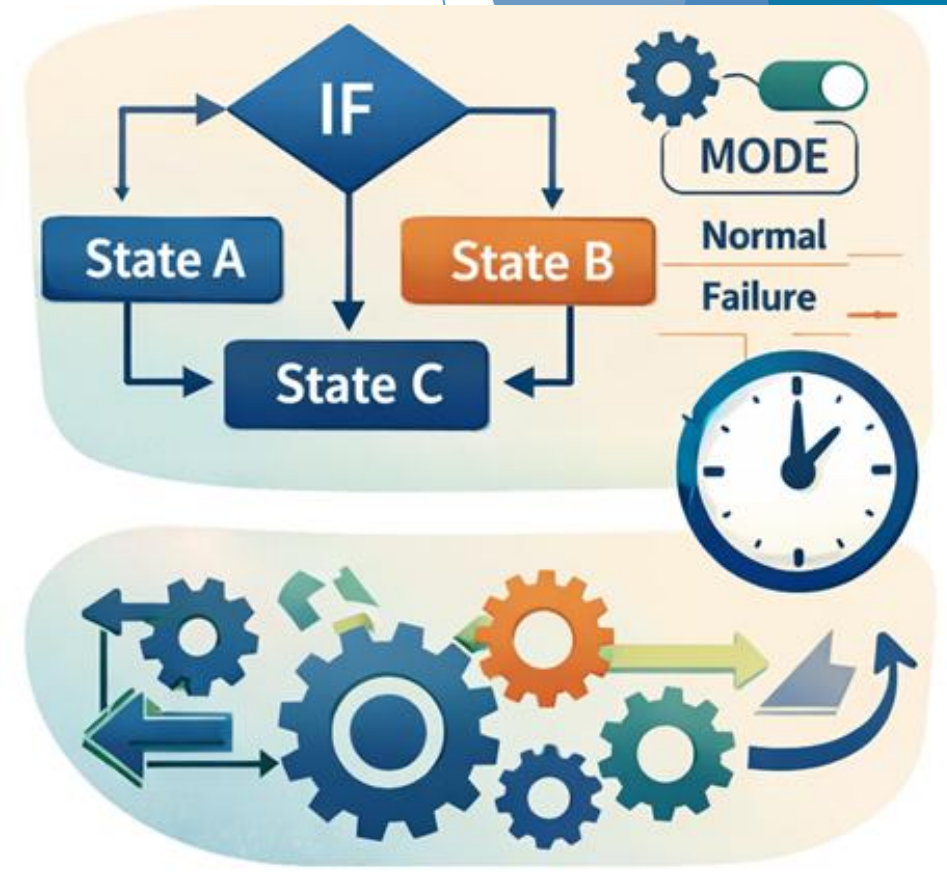
▶ Key Concepts:

- ▶ **Conditional execution** → if/then/else decisions per clock tick
- ▶ **State machines (automaton)** → model software behavior over time
 - ▶ Supports **hierarchical states** and transitions

▶ Characteristics:

- ▶ Synchronous with global clock ticks
- ▶ Deterministic and predictable execution
- ▶ Clear separation of states, modes and transitions

Control flow ensures structured, reliable decision-making in real-time systems



Swan Control-flow predefined operators

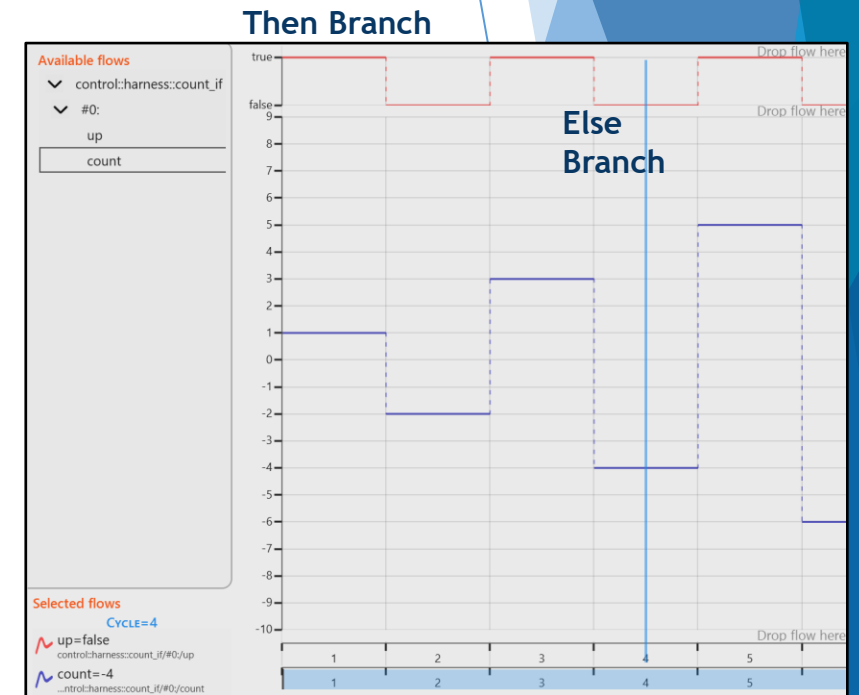
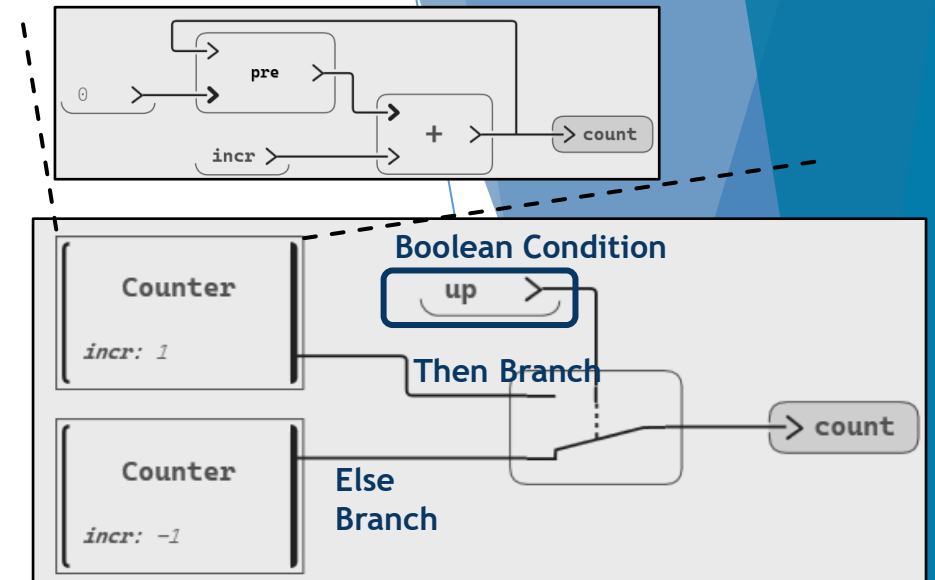
- ▶ The predefined operators are presented in the *User Documentation*:
 - ▶ selection operators (**if/then/else, case**)
 - ▶ **activate if/when** and
 - ▶ **automaton,**
 - ▶ **forward,**
 - ▶ **activate...default,**
 - ▶ **activate...last**

If-then-else Selection

- ▶ The **if-then-else** Block selects values depending on a Boolean condition

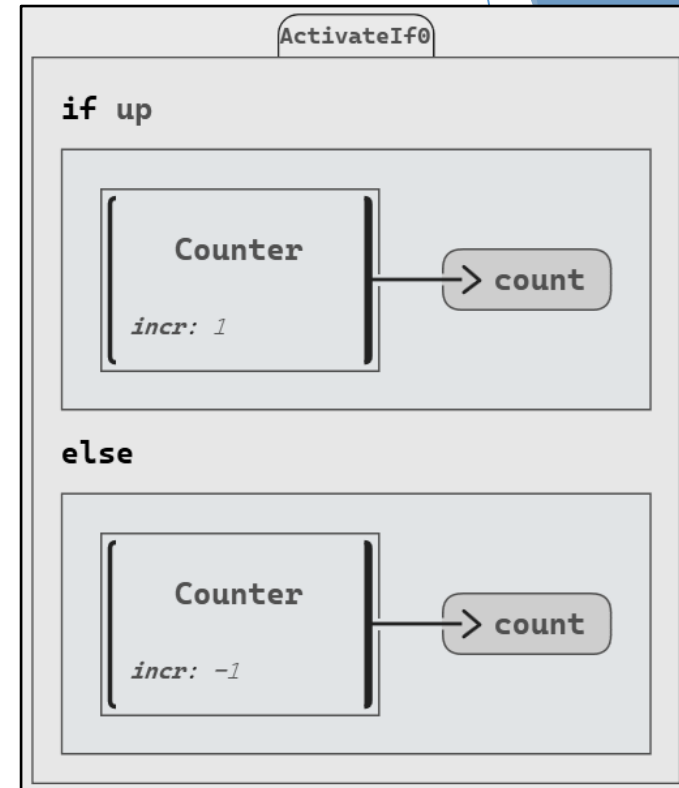
count = **if** up **then** Counter(1) **else** Counter(-1)

- ▶ When the condition is true, the if-then-else Block returns the value(s) evaluated in the **then** branch, otherwise the value(s) of the **else** branch
- ▶ The if-then-else operator **evaluates both branches at each time step** and returns the value according to the result of the condition evaluation
- ▶ The **Boolean condition** is on the **top**, the **first left pin** is the **then branch** and the **second left pin** is the **else branch**



When Do You Use an Activate If Block?

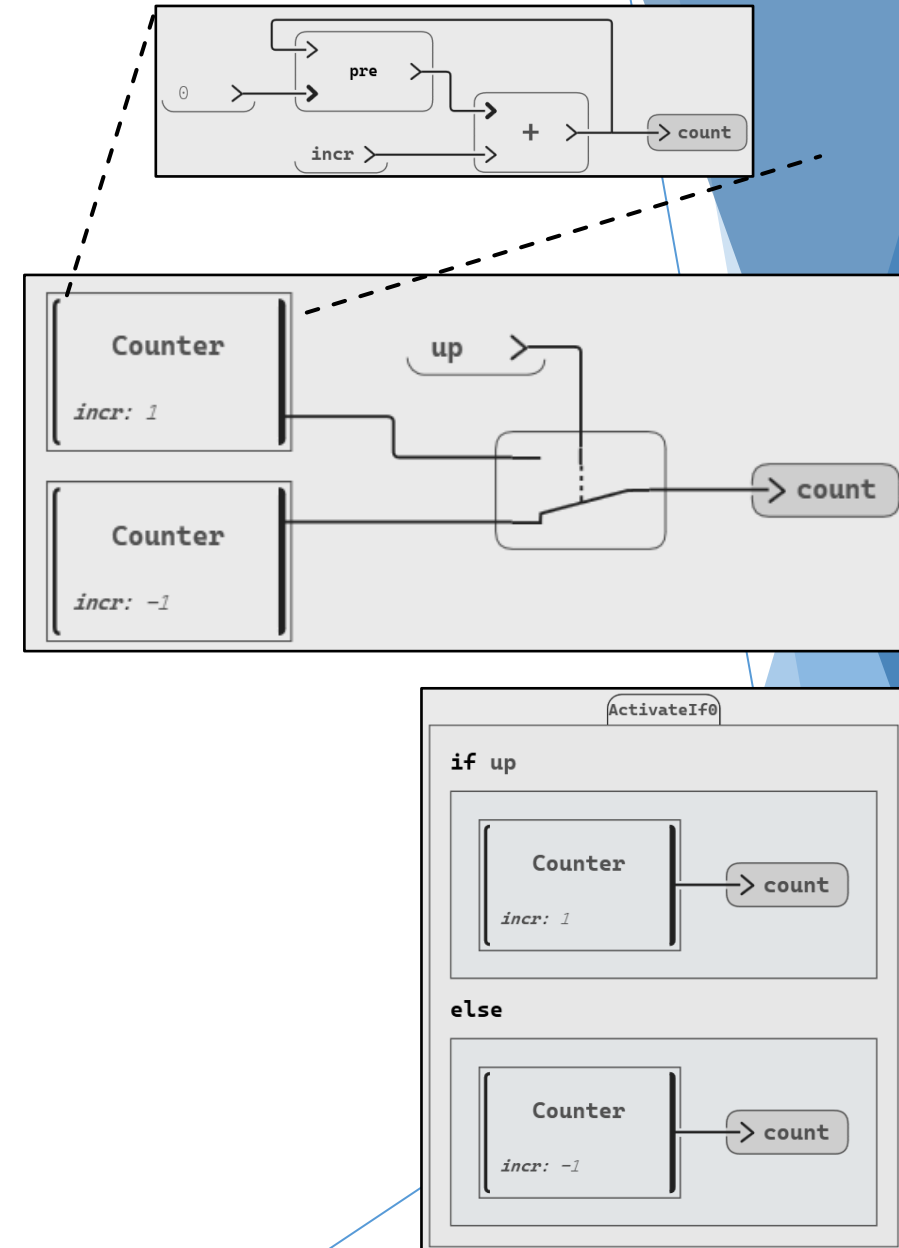
- ▶ In cases of control that correspond to situations where the current activation depends only on conditions computable in the current step (for example a button management)
- ▶ Swan introduces a simple decision mechanism by means of **Activate If** blocks, which from a semantics point of view are close to the Data-Flow activation condition, but with a more imperative flavor
- ▶ In the equational style, an **Activate If** declaration starts with the **activate if** keyword and the condition follows with the **then** keyword
- ▶ The branches can contain either another **Activate If** or a **scope** declaration.
- ▶ When a branch is active, the equations it contains are evaluated, they are not evaluated otherwise



```
activate $ActivateIf0 if up then {  
  let count = Counter(1);  
} else {  
  let count = Counter(-1);  
};
```

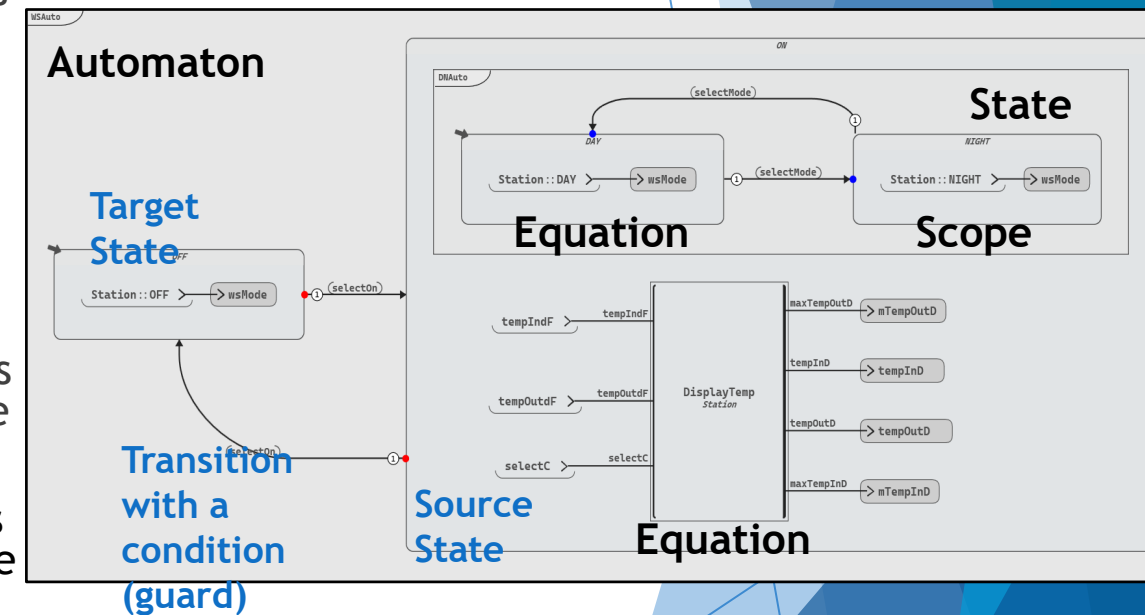
Activate If Versus If-Then-Else

- ▶ When an **Activate If** branch is active, the equations it contains are evaluated, they are not evaluated otherwise, while
- ▶ **If-Then-Else** branches are always evaluated at each step even if the branches are not active
- ▶ The **pre** operator yields the value produced in the previous step—either from the last step in which its branch was active (for *Activate If*), or from the immediately preceding step (for *If-Then-Else*).



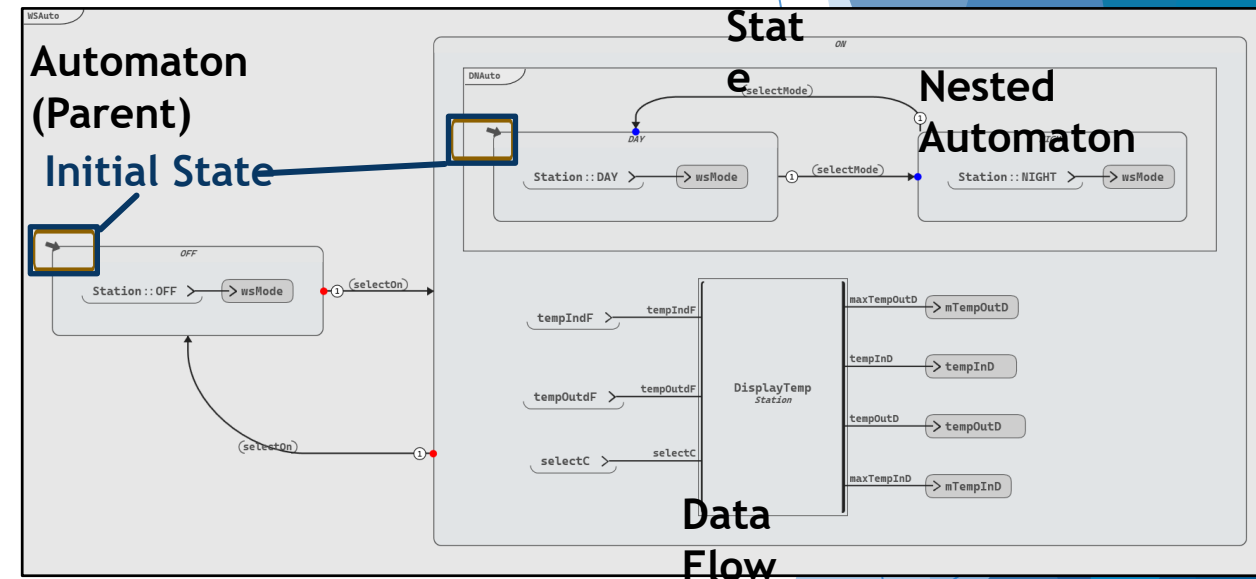
When Do You Use an Automaton (state machine)?

- ▶ An automaton is a very common way to introduce control structures in an application
- ▶ An automaton implements control flows to activate a mode. It performs actions depending on a sequence of several conditions and on the previous mode
- ▶ An automaton has a memory so:
 - ▶ The generated code always contains memories
 - ▶ Use it when you implement a mode management
- ▶ An automaton is a state machine where the activation depends on transition conditions (named **guards**) and the previous state
- ▶ A state machine is a collection of states, each state defines its own equation set where (at least) the output flows of the state machine are defined.
- ▶ Such a state machine can be called a **modal machine**: a state defines a mode where any flow defined by the state machine has its definition. The rules to determine which mode is applicable are defined by **transitions** between states and the previous state



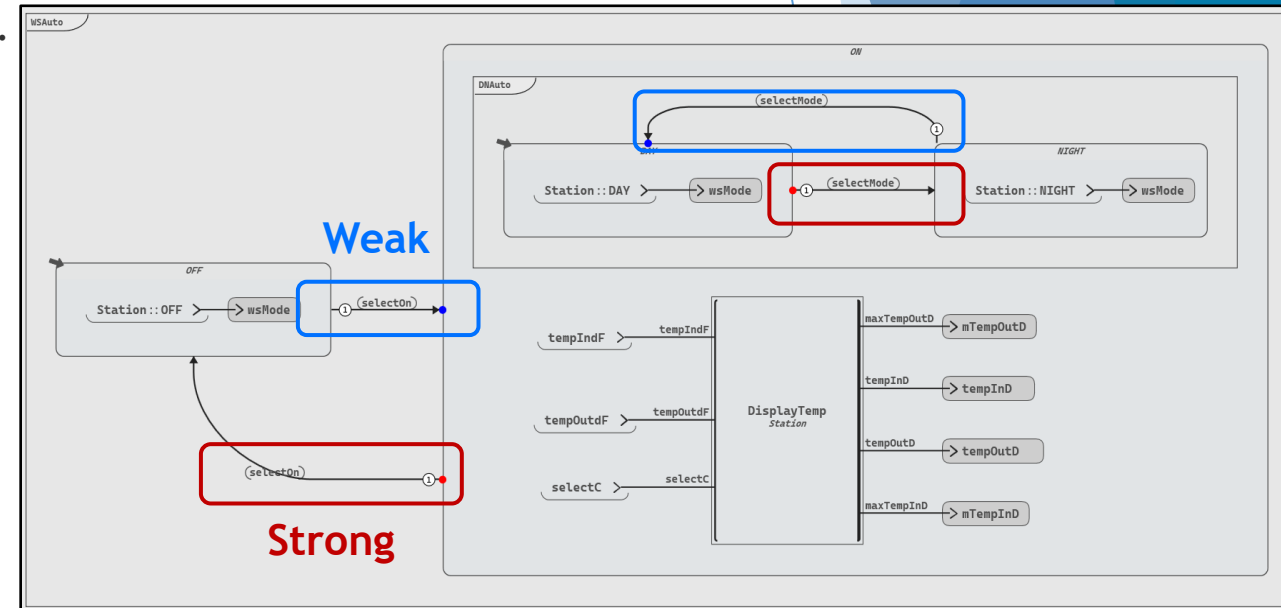
Automaton

- ▶ Per automaton
 - ▶ There is one and only one initial state (starting state at the first step)
 - ▶ There is one and only active state and at most one transition can be fired at each step
- ▶ Each flow defined by the automaton must have its first value defined in any state that can be active at the first step
- ▶ An automaton can be nested in another one via a state owned to the parent automaton
- ▶ An automaton can be mixed with data flows



Transitions in Automaton

- ▶ A transition starts from a **source state** to reach a **target state** under a condition, with eventual actions performed
- ▶ The condition (**guard**) is a **Boolean expression** like $a > 0$ or $a < 0$
 - ▶ When the condition is verified, the transition is fireable.
- ▶ A transition without guard is allowed in Swan:
 - ▶ In absence of guard, that is say **true** as guard, the transition is always fireable.
- ▶ It is possible to declare several outgoing transitions from a source state:
- ▶ The transitions in an automaton can be:
 - ▶ **Weak** (**until**) or
 - ▶ **Strong** (**unless**) or
 - ▶ Mixed **weak** and **strong**
- ▶ A transition can execute **actions** when its is fired, the actions being Boolean flow emissions or equation blocks written after *let* keyword.



Summary

- ▶ **Why tools matter:** improve reliability, enable early error detection, support standards, and manage complexity
- ▶ **Tool ecosystem:** requirements, architecture (UML/SysML), model-based design (SCADE), V&V, version control
- ▶ **Scade One:** model-based, formally defined, deterministic, with automatic code generation (correct-by-construction)
- ▶ **Swan language:** synchronous, deterministic, based on logical time (tick-based execution)
- ▶ **Dataflow paradigm:** systems modeled as networks of nodes; computation driven by data movement and dependencies
- ▶ **Key properties:** concurrency, determinism, synchronous execution, and clear data dependencies
- ▶ **Control flow paradigm:** complements dataflow using conditions and state machines (automata)
- ▶ **Core concepts:** operators (reusable blocks), strong typing, time operators (pre, ->), modular project structure

